Flutter Tutorial 7: Complete Overview of Default Project Structure (Files and Folders)

Introduction

In this tutorial, we will explore the complete structure of a Flutter project. When you create a Flutter app, many files and folders are generated automatically. Understanding these files is very important because it helps you know where to write code and how Flutter projects work internally.

Why Understanding Project Structure is Important

- Helps you locate files easily.
- Improves your development speed.
- Makes debugging easier.
- Helps you understand how Flutter manages apps for different platforms.

Main Project Folder

This is your root directory where the entire Flutter project exists.

Example:

`my_app_2`

All files and folders are inside this main project folder.

.dart_tool Folder

- This folder is automatically generated.
- It stores Flutter and Dart tool-related data.
- Used internally by Flutter for build and package management.

You usually do not need to modify anything inside this folder.

.idea Folder

- This folder is used by Android Studio and IntelliJ IDEA.
- It stores project settings and IDE configurations.

You should not manually edit this folder unless needed.

android Folder

- Contains all Android-specific code.
- Used when your app runs on Android devices.
- Includes Gradle files, manifests, and native Android code.

You only modify this folder when working with native Android features.

ios Folder

- Contains iOS-specific code.
- Used when running the app on iPhone or iPad.
- Includes Xcode project files.

Usually not modified unless working with iOS-specific features.

web Folder

- Contains files for running your app on the web.
- Includes HTML, CSS, and JavaScript integration.

Used when you build Flutter apps for browsers.

windows Folder

- Contains Windows desktop-specific code.
- Used for building apps for Windows OS.

linux Folder

- Contains Linux desktop-specific files.
- Used for building apps for Linux systems.

macos Folder

- Contains macOS-specific files.
- Used when building apps for Apple computers.

lib Folder (Most Important)

- This is the main folder where your Dart code is written.
- Contains the main file:

  `main.dart`

- This is the entry point of your application.

You will spend most of your time working inside this folder.

build Folder

- Stores compiled files and build outputs.
- Automatically generated when you run the app.

You should not edit this folder manually.

assets Folder

- Used to store images, fonts, icons, and other resources.

- You must declare assets inside `pubspec.yaml` to use them.

Example assets:

- Images
- Icons
- Fonts

test Folder

- Contains test files for your application.
- Used for unit testing and widget testing.

Testing helps ensure your app works correctly.

.gitignore File

- Used when working with Git.
- Specifies which files and folders should not be uploaded to Git repositories.

Example:

- build folder
- temporary files

.metadata File

- Stores Flutter project metadata.
- Used internally by Flutter tools.

No need to modify this file.

analysis_options.yaml File

- Used to define lint rules and code analysis settings.
- Helps maintain clean and consistent code.

pubspec.yaml File (Very Important)

- This is one of the most important files in a Flutter project.
- Used to manage:
    - Project name
    - Dependencies (packages)
    - Assets
    - Fonts

Example:

- Adding packages
- Declaring assets

pubspec.lock File

- Automatically generated file.
- Stores exact versions of installed packages.

Helps maintain consistency across systems.

README.md File

- Contains information about your project.
- You can write project description, setup instructions, and usage.

.iml File

- Used by IntelliJ-based IDEs.
- Stores module configuration.

External Libraries Section

- Shows all installed packages and dependencies.
- Helps you access third-party libraries.

Scratches and Consoles

- Temporary workspace for testing code.
- Useful for quick experiments.

Summary of Important Files

- `lib/main.dart` → Entry point of app
- `pubspec.yaml` → Manage dependencies and assets
- `assets/` → Store resources
- `android/`, `ios/` → Platform-specific code

Conclusion

Understanding the Flutter project structure is very important for every developer. It helps you work efficiently and understand where everything belongs. Most of your development work will be inside the lib folder, while other folders handle platform-specific and system-level operations.

End of Flutter Tutorial 7